HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BỘ MÔN IOT VÀ ỨNG DỤNG



BÁO CÁO BÀI TẬP LỚN

Giảng viên hướng dẫn : TS. Nguyễn Quốc Uy

Nhóm : 10

Họ và tên sinh viên	Mã sinh viên
ĐINH GIA KHÁNH	B23DCCN424

I. GIỚI THIỆU VỀ HỆ THỐNG 3

II. SRS 4

Use case 4

Lưu dữ liệu 4

Hiển thị dữ liệu 5

Bật tắt đèn 8

Hiện thị cảm biến 10

Hiển thị lịch sửa trạng thái và xuất báo cáo dữ liệu 12

Lược đồ quan hệ 17

I. GIỚI THIỆU VỀ HỆ THỐNG

Hệ thống là giải pháp phần mềm toàn diện nhằm thu thập, giám sát các chỉ số môi trường theo thời gian thực và tự động hóa việc điều khiển các thiết bị ngoại vi. Hệ thống hướng đến mục tiêu tối ưu hóa vận hành, hỗ trợ ra quyết định nhanh chóng và số hóa việc quản lý dữ liệu thông qua giao diện trực quan, thân thiện với người dùng.

Kiến trúc & Công nghệ cốt lõi:

Tầng thu thập & Kết nối (IoT Layer):

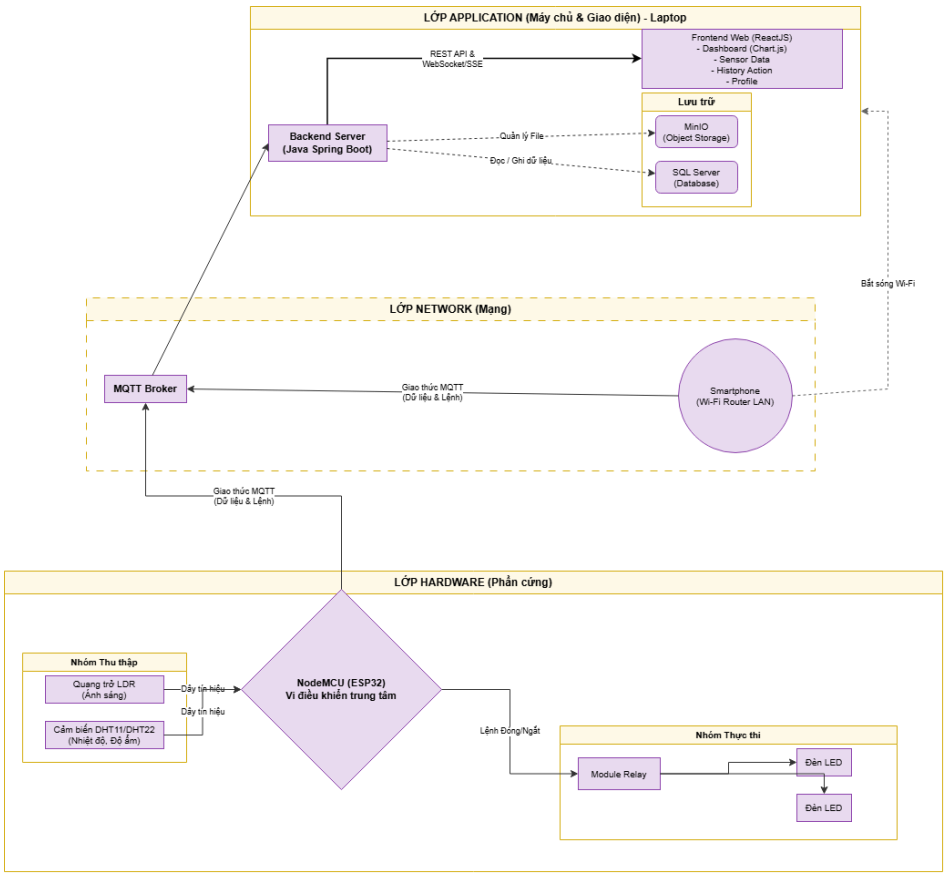
Tích hợp các cảm biến môi trường (nhiệt độ, độ ẩm không khí, cường độ ánh sáng...) và các cơ cấu chấp hành (relay điều khiển đèn, thiết bị phụ tải).

Giao thức truyền thông thời gian thực chuẩn công nghiệp **MQTT**, đảm bảo độ trễ thấp, kết nối ổn định và tiết kiệm băng thông.

Tầng ứng dụng & Trực quan hóa:

Giao diện hiện đại trên nền Dark Mode tối ưu trải nghiệm theo dõi liên tục.

Phản hồi trạng thái hai chiều giữa lệnh điều khiển từ người dùng và thiết bị thực tế.



Hình 1: Thiết kế tổng thể

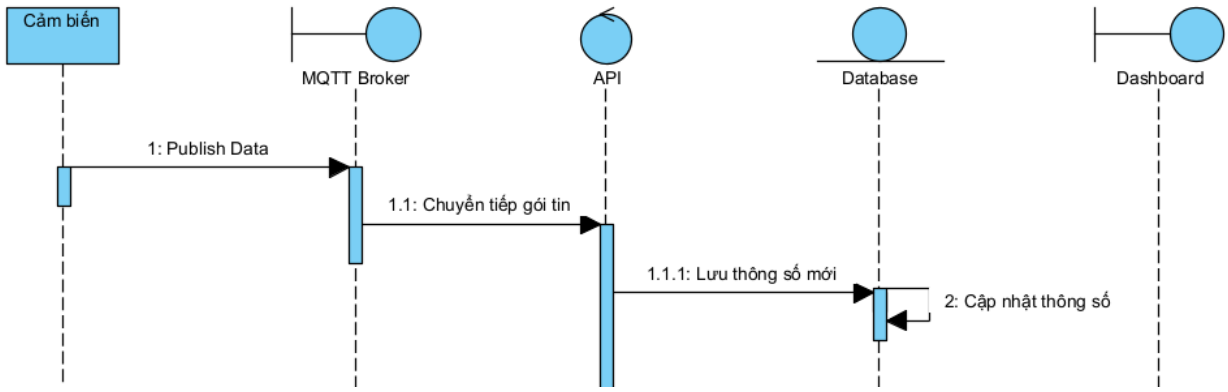
II. SRS

Use case

Lưu dữ liệu

Luồng thu thập, lưu trữ theo thời gian

Sequence Diagram



1: Publish Data: Thiết bị IoT định kỳ đọc giá trị từ các cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng) và đóng gói dạng JSON rồi gửi (Publish) lên MQTT Broker qua topic iot/sensors/telemetry.

1.1: Chuyển tiếp gói tin: MQTT Broker nhận gói tin và chuyển tiếp (Forward) ngay lập tức đến API Backend / Worker Service (đang đăng ký lắng nghe topic này).

1.1.1 & 1.1.1.1: Lưu & Cập nhật cơ sở dữ liệu:

Backend bóc tách (parse) dữ liệu và thực thi lệnh thêm mới bản ghi đo đạc vào bảng sensor_data để lưu trữ lịch sử phục vụ vẽ biểu đồ.

Đồng thời thực hiện câu lệnh cập nhật giá trị tức thời (current_value, current_status) vào bảng danh mục sensors.

API:

Tiếp nhận dữ liệu cảm biến định kỳ (HTTP Ingestion)

Endpoint: POST /api/v1/sensors/telemetry

Mô tả: Nhận dữ liệu đo đạc định kỳ từ phần cứng/Gateway để lưu ngầm vào bảng sensor_data bất kể người dùng có mở giao diện hay không.

Request Body:

```
{
  "nodeId": "ESP32_STATION_01",
  "readings": [
    { "sensorId": 1, "sensorType": "Temperature", "value": 26.5 },
```

{ "sensorId": 2, "sensorType": "Humidity", "value": 60.0 },
{ "sensorId": 3, "sensorType": "Light intensity", "value": 520.0 }
]
}
Response (201 Created):
{
"statusCode": 201,
"message": "Telemetry data ingested successfully",
"data": {
"recordsInserted": 3,
"recordedAt": "2026-09-09T06:45:00Z"
}
}

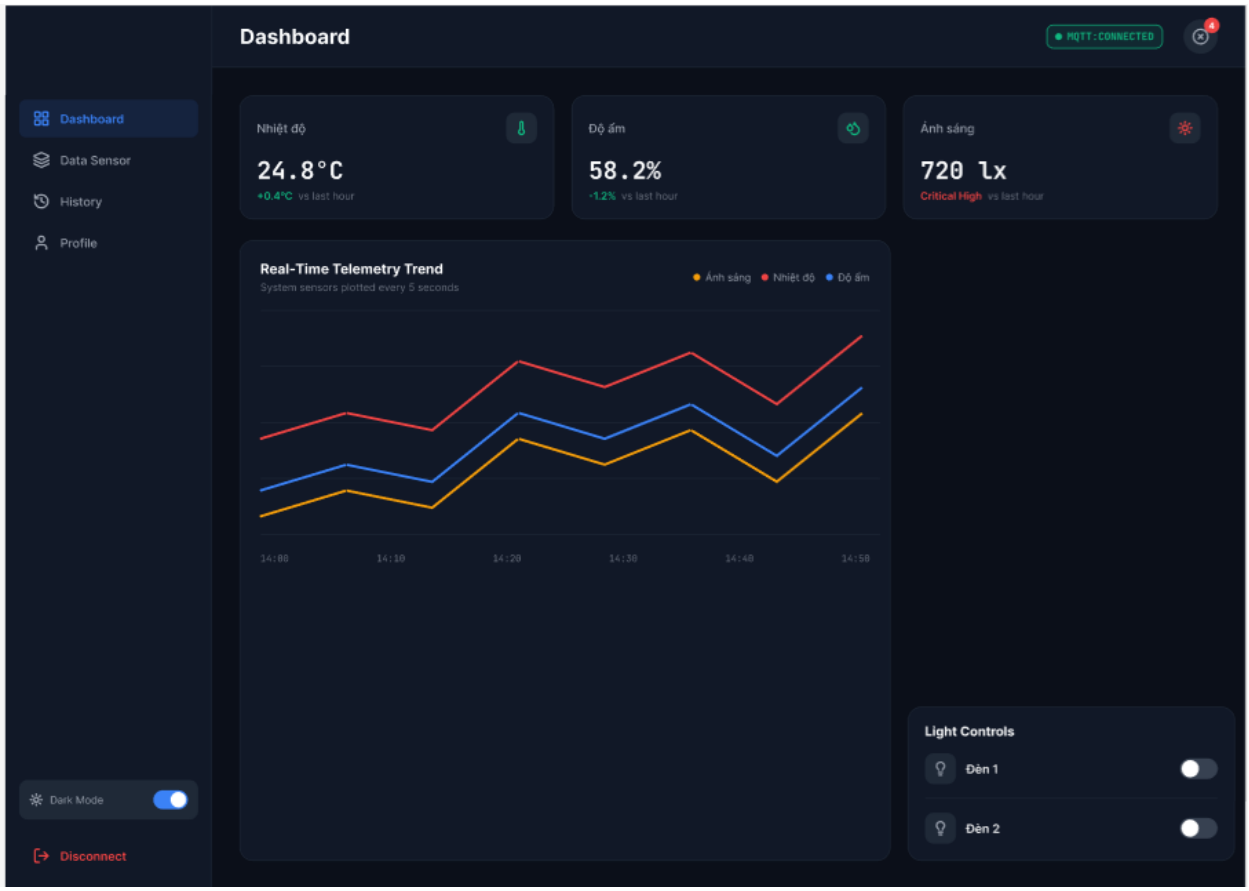
- **Đặc tả MQTT Inbound (Kênh nạp ngầm chính qua Broker)**
  - **Broker Host: tcp://broker.hivemq.com:1883**
  - **Topic: iot/sensors/telemetry**
  - **QoS: 1**
  - **Payload:**

JSON
{
"station": "ESP32_01",
"data": {
"temperature": 26.5,
"humidity": 60.0,
"light": 520.0
}
}

Hiển thị dữ liệu

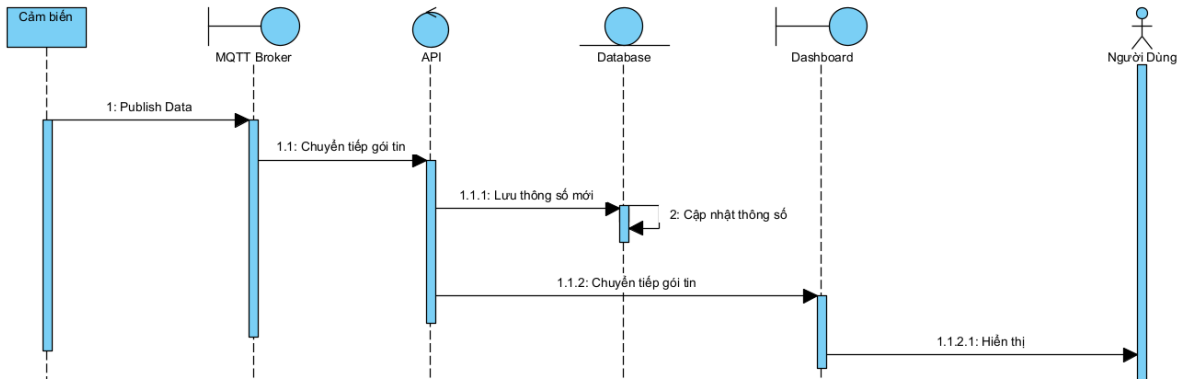
Luồng thu thập, lưu trữ và hiển thị dữ liệu cảm biến thời gian

Giao diện:



Hình 2.1: Giao diện Dashboard

Sequence Diagram



1: Publish Data: Thiết bị IoT định kỳ đọc giá trị từ các cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng) và đóng gói dạng JSON rồi gửi (Publish) lên MQTT Broker qua topic iot/sensors/telemetry.

1.1: Chuyển tiếp gói tin: MQTT Broker nhận gói tin và chuyển tiếp (Forward) ngay lập tức đến API Backend / Worker Service (đang đăng ký lắng nghe topic này).

1.1.1 & 1.1.1.1: Lưu & Cập nhật cơ sở dữ liệu:

Backend bóc tách (parse) dữ liệu và thực thi lệnh thêm mới bản ghi đo đạc vào bảng sensor_data để lưu trữ lịch sử phục vụ vẽ biểu đồ.

Đồng thời thực hiện câu lệnh cập nhật giá trị tức thời (current_value, current_status) vào bảng danh mục sensors.

1.1.2: Gửi dữ liệu: Backend đẩy gói tin thời gian thực qua giao thức WebSocket tới giao diện người dùng (DASHBOARD).

1.1.2.1: Hiển thị: DASHBOARD nhận dữ liệu từ WebSocket, lập tức cập nhật lại các đồng hồ đo (Gauge), biểu đồ đường và bảng dữ liệu mà không cần người dùng phải tải lại (Reload/F5) trang web.

API:

Lấy dữ liệu khởi tạo khi vừa mở Web:

Endpoint: GET /api/v1/dashboard/metrics

Description: Trả về các giá trị nhiệt độ, độ ẩm, ánh sáng tức thời và xu hướng tăng/giảm.

Response (200 OK):

```
{
  "temperature": { "value": 24.8, "unit": "°C", "trend": "+0.4" },
  "humidity": { "value": 58.2, "unit": "%", "trend": "-1.2" },
  "light": { "value": 720, "unit": "lx", "status": "CRITICAL_HIGH" }
}
```

Lấy dữ liệu vẽ biểu đồ xu hướng

Endpoint: GET /api/v1/dashboard/telemetry-trend

Query Params: range=24h, interval=5m

Description: Trả về danh sách tọa độ thời gian thực để vẽ Line Chart.

Response (200 OK):

```
[
  { "time": "14:00", "temp": 24.0, "humidity": 55.0, "light": 650 },
  { "time": "14:10", "temp": 24.8, "humidity": 58.2, "light": 720 }
]
```

Kênh truyền dữ liệu thời gian thực (WebSocket)

Endpoint (WS): ws://localhost:8080/ws/live-data

Subscription Topic: /topic/sensors

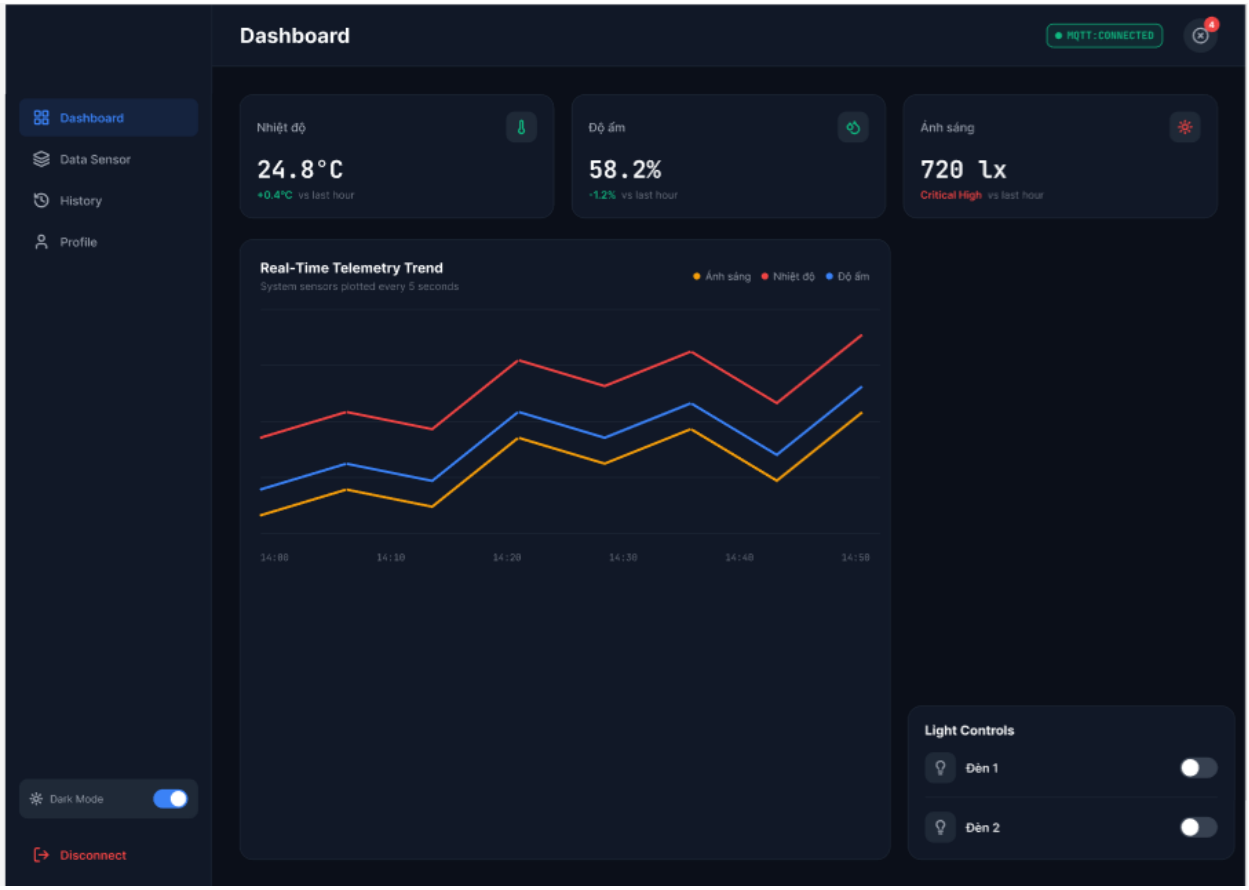
Payload Broadcast:

```
{
  "event": "SENSOR_UPDATE",
  "timestamp": "14:32:48",
  "sensors": [
    { "id": "SEN-01", "value": 24.8, "unit": "°C", "status": "OPTIMAL" },
    { "id": "SEN-02", "value": 58.2, "unit": "%", "status": "OPTIMAL" },
    { "id": "SEN-03", "value": 720.0, "unit": "lx", "status": "CRIT_HIGH" }
  ]
}
```


Bật tắt đèn

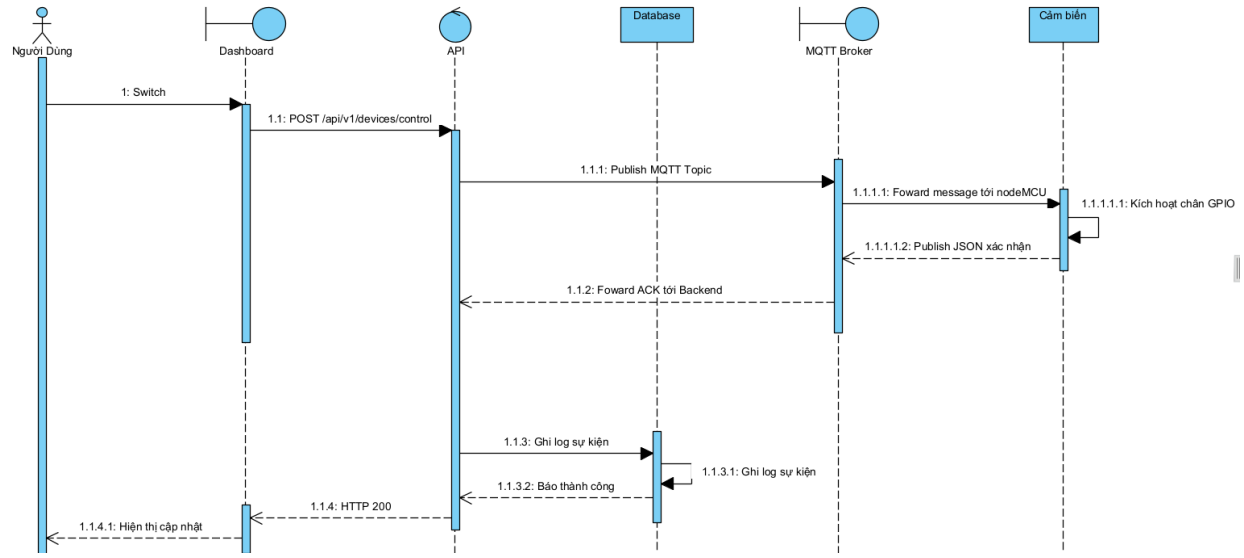
Luồng điều khiển bật/tắt thiết bị ngoại vi và ghi log trạng thái

Giao diện



Hình 2.2: Giao diện Dashboard

Sequence Diagram



1: Switch: Người dùng thao tác gạt nút chuyển đổi trạng thái (Toggle Switch) của thiết bị (VD: Đèn 1) trên giao diện Dashboard.

1.1: POST /api/v1/devices/control: Giao diện Dashboard gửi HTTP POST request đến API Backend mang theo payload JSON chứa ID thiết bị và trạng thái mới ({ "deviceId": 1, "action": "TURN_ON", "state": "ON" }).

1.1.1: Publish MQTT Topic: API Backend tiếp nhận yêu cầu, đóng gói lệnh điều khiển và Publish gói tin lên MQTT Broker theo Topic điều khiển tương ứng của thiết bị (iot/device/1/command).

1.1.1.1: Forward message tới NodeMCU: MQTT Broker tiếp nhận bản tin và định tuyến chuyển tiếp (forward) lệnh xuống vi điều khiển NodeMCU (ESP32) đang kết nối mạng.

1.1.1.1.1: Kích hoạt chân GPIO: NodeMCU (ESP32) nhận lệnh, xử lý mức logic và kích chân GPIO tương ứng để đóng/ngắt Module Relay, thay đổi trạng thái hoạt động thực tế của thiết bị phần cứng.

1.1.1.1.2: Publish JSON xác nhận: Sau khi kích hoạt chân GPIO thành công, NodeMCU (ESP32) đóng gói bản tin xác nhận (ACK) dạng JSON và Publish ngược lại lên MQTT Broker qua Topic trạng thái (iot/device/1/status).

1.1.2: Forward ACK tới Backend: MQTT Broker chuyển tiếp bản tin xác nhận (ACK) từ vi điều khiển về cho API Backend.

1.1.3: Ghi log sự kiện: Sau khi nhận tín hiệu xác nhận thành công từ phần cứng, API Backend gửi lệnh truy vấn xuống Database (SQL Server).

1.1.3.1: Ghi log sự kiện: Database (SQL Server) thực thi:

- Thêm bản ghi mới vào bảng action_history (lưu vết user_id, device_id, action, trigger_by, status, created_at).
- Cập nhật trạng thái hoạt động mới nhất (current_status = 'ON') vào bảng devices.

1.1.3.2: Báo thành công: Database (SQL Server) phản hồi xác nhận giao dịch ghi dữ liệu và cập nhật trạng thái đã hoàn tất thành công cho API Backend.

1.1.4: HTTP 200: API Backend đóng gói dữ liệu và gửi phản hồi HTTP Status 200 (OK) kèm trạng thái thiết bị vừa cập nhật về cho giao diện Dashboard.

1.1.4.1: Hiển thị cập nhật: Giao diện Dashboard nhận phản hồi, cập nhật hiệu ứng nút gạt (Switch) sang màu xanh (Active/ON) và hiển thị thông báo thành công cho Người Dùng.

API:

RESTful API từ Giao diện gửi lên Backend

Endpoint: POST /api/v1/devices/control

Request Body:


```
{
  "deviceId": "01",
  "action": "TURN_ON",
  "state": "ON"
}
```

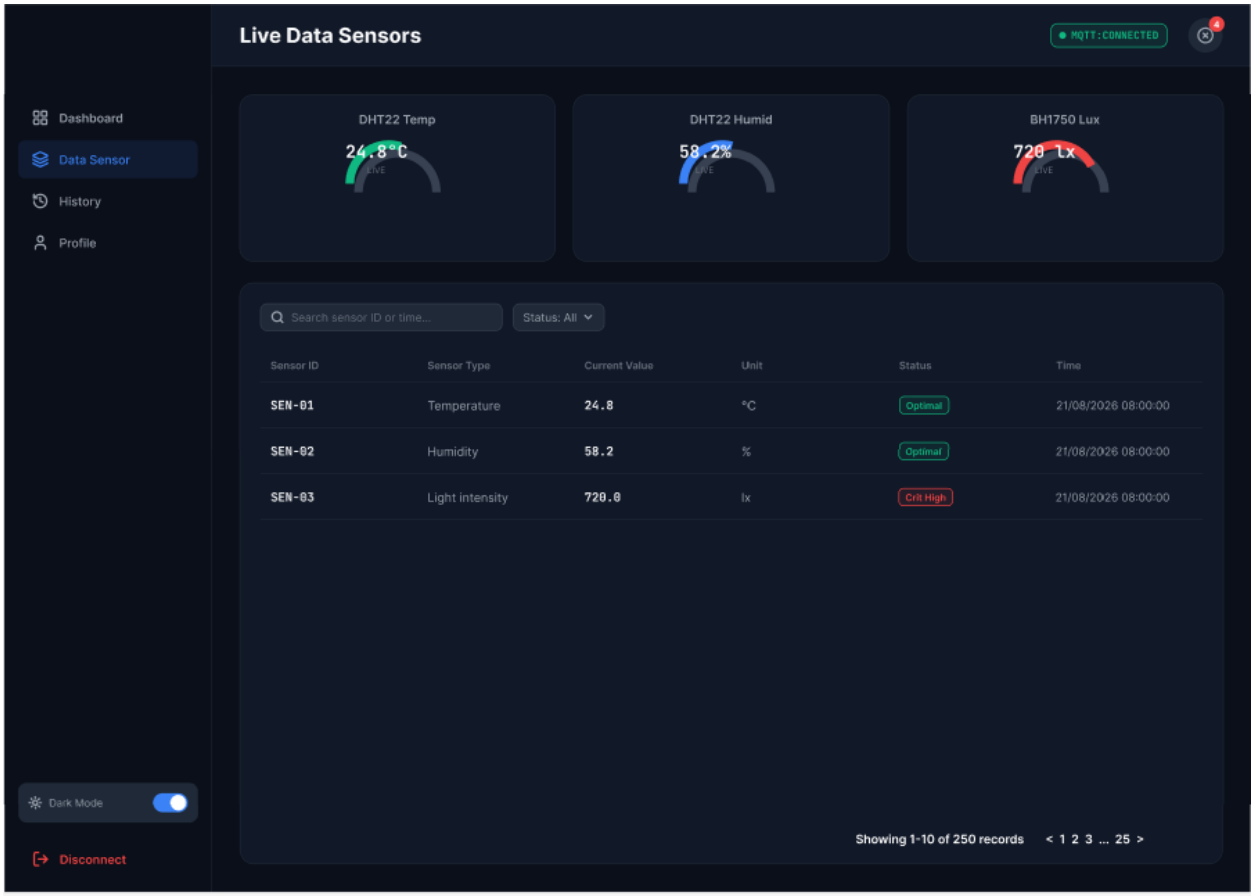

Response (200 OK):


```
{
  "statusCode": 200,
  "message": "Device state updated successfully",
  "data": {
    "deviceId": "01",
    "status": "ON",
    "updatedAt": "2026-08-26T14:32:48Z"
  }
}
```


Hiện thị cảm biến

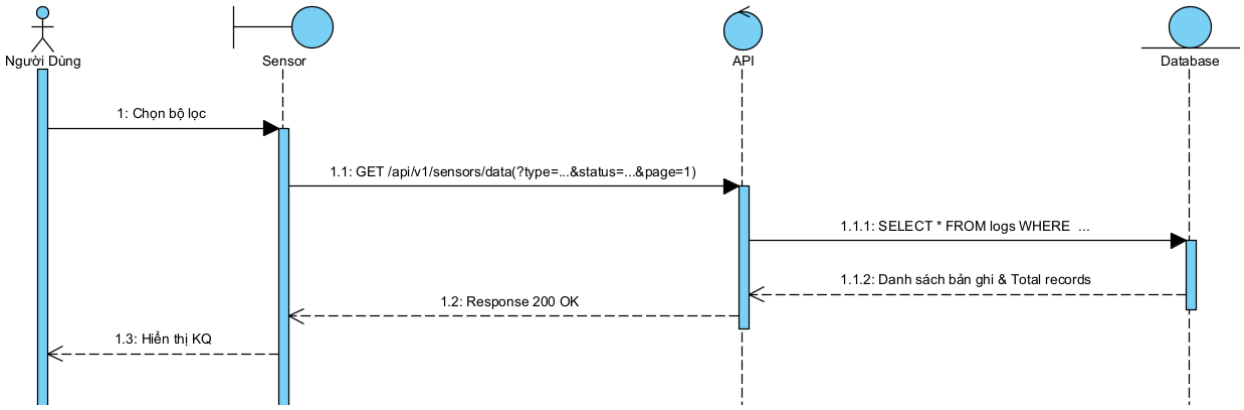
Luồng tìm kiếm và lọc dữ liệu cảm biến trên trang Data Sensor

Giao diện



Hình 2.3: Giao diện Data sensor

Sequence Diagram



1: Chọn bộ lọc) Người dùng thao tác chọn các điều kiện lọc (loại cảm biến, trạng thái hoặc nhập từ khóa tìm kiếm) trên giao diện Data Sensor (UI - đối tượng Sensor trong sơ đồ).

1.1: Gửi yêu cầu: Giao diện gửi HTTP GET request chứa các thông số điều kiện lọc (query params) tới API Backend.

1.1.1: Lấy thông số: API Backend tiếp nhận tham số, xây dựng câu lệnh truy vấn SQL (Query) phù hợp và gửi yêu cầu tới Database.

1.1.2: Trả về thông số: Database thực thi truy vấn và trả về danh sách các bản ghi thông số cảm biến thỏa mãn điều kiện lọc cho API Backend.

1.2: Trả về thông số: API Backend chuẩn hóa dữ liệu thành định dạng JSON và gửi phản hồi HTTP Status 200 (OK) về cho giao diện Data Sensor (UI).

1.3: Hiển thị: Giao diện Data Sensor (UI) nhận phản hồi và cập nhật lại danh sách dữ liệu trên bảng hiển thị cũng như các đồng hồ đo (Gauges) cho Người Dùng theo dõi.

API:

API lọc thông số cảm biến

Endpoint: GET /api/v1/sensors/filter

Query Params: status=OPTIMAL, type=Temperature, search=SEN-01

Description: Lấy trạng thái, giá trị tức thời của từng cảm biến theo điều kiện lọc.

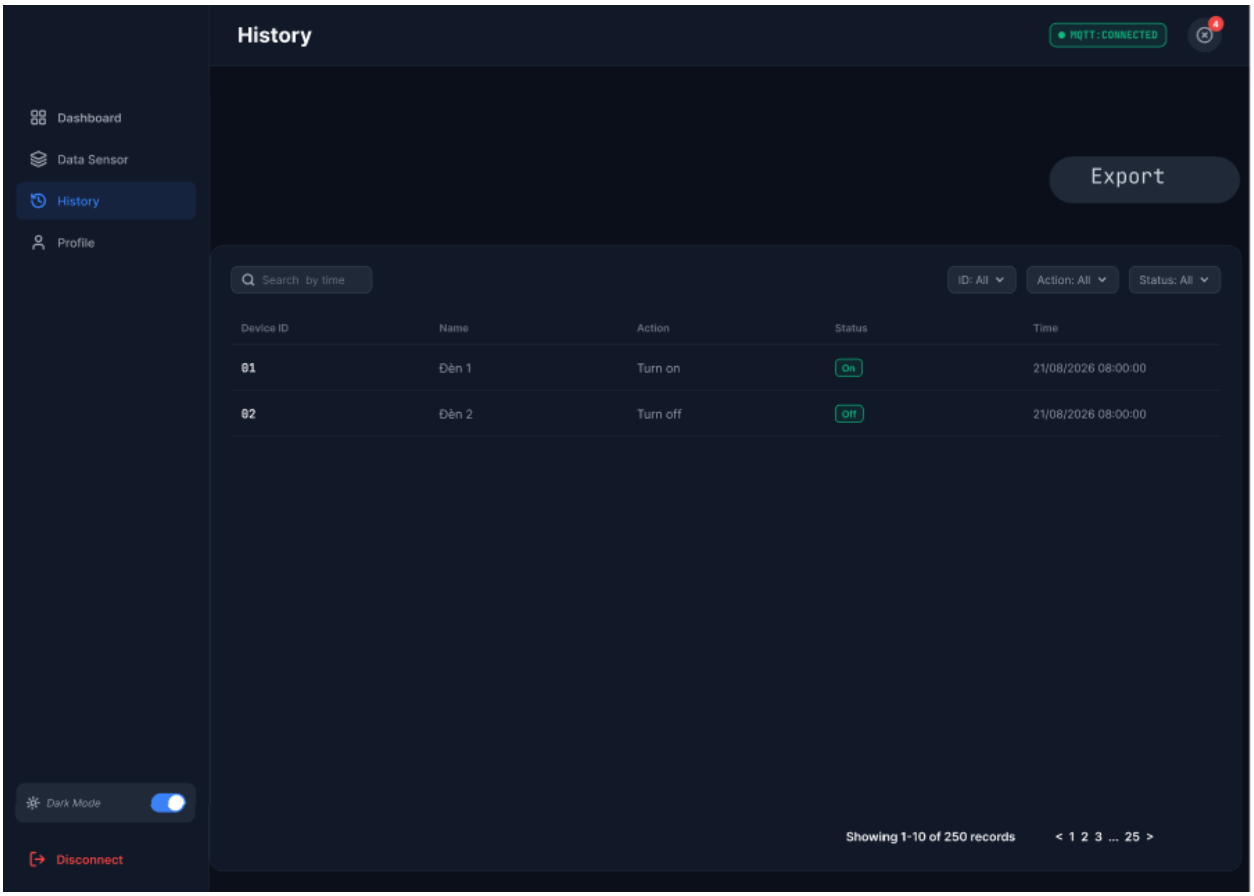
Response (200 OK):


```
{
  "statusCode": 200,
  "totalRecords": 1,
  "data": [
    {
      "sensorId": "SEN-01",
      "name": "DHT22 Temp",
      "type": "Temperature",
      "currentValue": 24.8,
      "unit": "°C",
      "status": "OPTIMAL",
      "lastUpdated": "2026-08-26T14:32:48Z"
    }
  ]
}
```


Hiển thị lịch sửa trạng thái và xuất báo cáo dữ liệu

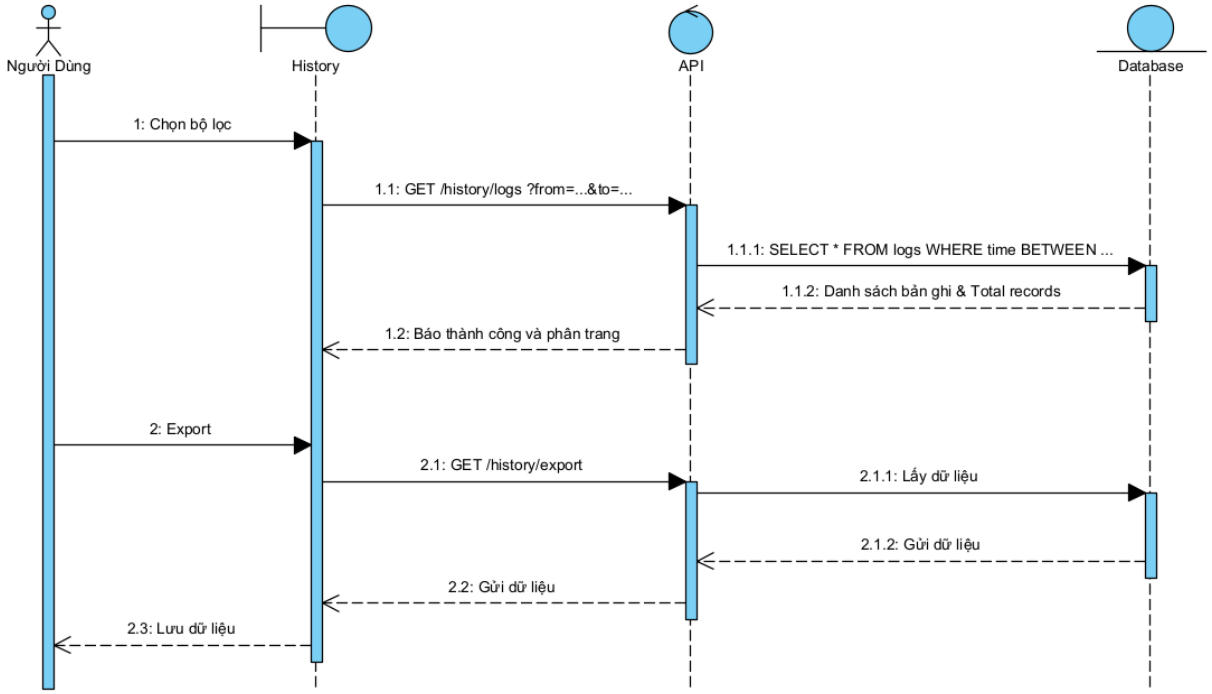
Luồng truy vấn, phân trang lịch sử hoạt động và xuất báo cáo dữ liệu

Giao diện



Hình 2.4: Giao diện trang History

Sequence Diagram



1: Chọn bộ lọc: Người dùng chọn khoảng thời gian (Từ ngày - Đến ngày) và nhập từ khóa tìm kiếm trên giao diện History.

1.1: GET /history/logs ?from=...&to=...: Giao diện History gửi HTTP GET request kèm các tham số bộ lọc và phân trang tới API.

1.1.1: SELECT * FROM logs WHERE time BETWEEN ...: API tiếp nhận yêu cầu, xây dựng câu lệnh SQL truy vấn và gửi tới Database để lọc dữ liệu cùng tổng số bản ghi.

1.1.2: Danh sách bản ghi & Total records: Database thực thi truy vấn và trả về danh sách lịch sử thỏa mãn điều kiện cho API.

1.2: Báo thành công và phân trang: API chuẩn hóa dữ liệu thành định dạng JSON và gửi phản hồi HTTP Status 200 (OK) về cho giao diện History để hiển thị danh sách kèm thanh phân trang cho Người Dùng.

1.3: Hiện thị dữ liệu ra màn hình cho người dùng

2: Export: Người dùng bấm nút xuất báo cáo trên giao diện History.

2.1: GET /history/export: Giao diện History gửi HTTP GET request yêu cầu xuất file báo cáo kèm điều kiện lọc tới API.

2.1.1: Lấy dữ liệu: API gửi câu lệnh truy vấn tới Database để lấy toàn bộ dữ liệu lịch sử theo điều kiện lọc.

2.1.2: Gửi dữ liệu: Database trả về tập dữ liệu thô cho API để tiến hành khởi tạo luồng dữ liệu file (File Stream CSV/Excel).
2.2: Gửi dữ liệu: API trả về luồng dữ liệu file đính kèm tiêu đề tải xuống cho giao diện History.
2.3: Lưu dữ liệu: Giao diện History tự động kích hoạt trình duyệt lưu file báo cáo về thiết bị của Người Dùng.
2.4: Giao diện thông báo lưu file thành công cho người dùng
API:
Truy vấn lịch sử hoạt động & Phân trang
Endpoint: GET /api/v1/history/logs
Query Params:
from: 2026-08-20T08:00:00Z
to: 2026-08-21T08:00:00Z
page: 1
size: 10
search: 01
Response (200 OK):


```
{
  "statusCode": 200,
  "data": {
    "totalRecords": 250,
    "page": 1,
    "size": 10,
    "totalPages": 25,
    "logs": [
      {
        "logId": 101,
        "deviceId": "01",
        "deviceName": "Đèn 1",
        "action": "Turn on",
        "status": "ON",
        "triggeredBy": "Admin",
        "timestamp": "2026-08-21T08:00:00Z"
```

```
    },
    {
      "logId": 102,
      "deviceId": "02",
      "deviceName": "Đèn 2",
      "action": "Turn off",
      "status": "OFF",
      "triggeredBy": "System_Schedule",
      "timestamp": "2026-08-21T08:00:00Z"
    }
  ]
 }
}
```


Xuất báo cáo lịch sử (Export File)

Endpoint: GET /api/v1/history/export

Query Params: from=2026-08-20T08:00:00Z, to=2026-08-21T08:00:00Z, format=csv

Response Headers:

Content-Type: text/csv; charset=utf-8

Content-Disposition: attachment; filename="history_report.csv"

Response Body: File binary stream (.csv / .xlsx).

Lược đồ quan hệ

